Furuno DRS4D-NXT Radar

Wire Protocol & System Technical Manual

J.C. Vaught

March 2026

A complete technical reference for the Furuno DRS4D-NXT marine radar system, covering the wire protocol, echo data encoding, system architecture, web interface, and development guide.

Contents

I Overview & Usage **2**

1 Introduction **2**
1.1 Hardware Overview . . . 2
1.2 Software Components . . . 2
1.3 Reverse Engineering Context . . . 3

2 Quick Start & Installation **4**
2.1 Prerequisites . . . 4
2.2 Building . . . 4
2.3 First Run . . . 4
2.4 Auto-Discovery . . . 4

3 CLI Modes **6**
3.1 General Syntax . . . 6
3.2 Web Server Mode . . . 6
3.3 Discover Mode . . . 6

4 Web Interface **7**
4.1 Starting the Server . . . 7
4.2 Page Tour . . . 7
4.2.1 Controls (index.html) . . . 7
4.2.2 Radar View (radar.html) . . . 7
4.2.3 Settings (settings.html) . . . 7
4.2.4 System Status (status.html) . . . 7
4.3 PPI Display . . . 7

5 Recording & Data Formats **8**
5.1 Starting and Stopping Recording . . . 8
5.2 Output Directory Structure . . . 8
5.3 CSV Format . . . 8
5.4 Settings Snapshot . . . 8

6 Pattern System **9**
6.1 What Patterns Do . . . 9
6.2 Simple Pattern Format (7-Field) . . . 9
6.3 Advanced Pattern Format (13-Field) . . . 9
6.4 Pattern Queue . . . 9
6.5 Loading Patterns . . . 9
6.6 NaN Override Behavior . . . 9

II Software Internals **10**

7 System Architecture Overview **10**

7.1 Module Summary . . . 10
7.2 System-Level Block Diagram . . . 10
7.3 Module-Level Diagram . . . 10
7.4 Data Flow Diagram . . . 10
7.5 Concurrency Model . . . 10

8 PPI Rendering **11**
8.1 Coordinate Transformation . . . 11
8.2 Color Mapping . . . 11
8.3 Overlay Elements . . . 11
8.4 PNG Encoding . . . 11
8.5 How to Modify . . . 11

9 Web Server **12**
9.1 Framework . . . 12
9.2 Shared State . . . 12
9.3 Route Registration . . . 12
9.4 Handler Pattern . . . 12
9.5 Echo Listener Thread . . . 12
9.6 Static File Serving . . . 12
9.7 How to Add a New Endpoint . . . 12

10 REST API Reference **13**
10.1 Power and Connection . . . 13
10.2 Settings . . . 13
10.3 State Queries . . . 13
10.4 Echo Image . . . 13
10.5 Recording . . . 13
10.6 Pattern Execution . . . 13
10.7 Stub Endpoints . . . 13

11 Recording System Internals **14**
11.1 Echo Listener Integration . . . 14
11.2 Directory Creation . . . 14
11.3 CSV Writing . . . 14
11.4 Settings Snapshots . . . 14
11.5 Setting Change History . . . 14

12 Pattern Execution Engine **15**
12.1 Step Parsing . . . 15
12.2 Differential Command Generation . . . 15
12.3 Single Pattern Execution . . . 15
12.4 Queue Execution . . . 15
12.5 NaN Override Logic . . . 15
12.6 Stop Mechanism . . . 15

13 CLI Modes (All Other Modes) **16**
13.1 Handshake . . . 16
13.2 TX On / TX Off . . . 16

13.3 Full Cycle . . . 16
13.4 Render Live . . . 16
13.5 Echo Listen . . . 16
13.6 Command Test . . . 16
13.7 Dump Raw . . . 16
13.8 Test Commands . . . 16
13.9 Validate . . . 16

14 Connection State Machine **17**
14.1 States and Transitions . . . 17
14.2 RadarError Enum . . . 17
14.3 Timeout Values . . . 17
14.4 Graceful Degradation . . . 17
14.5 Keepalive Handling . . . 17

15 Testing & Validation **18**
15.1 Running Tests . . . 18
15.2 Test Categories . . . 18
15.2.1 Connection Tests . . . 18
15.2.2 TX Tests . . . 18
15.2.3 Settings Tests . . . 18
15.2.4 Echo Impact Tests . . . 18
15.2.5 Render Integration Test . . . 18
15.3 A/B Command Testing . . . 18
15.4 Validation Suite . . . 18

III Echo Data Pipeline **19**

16 Echo Data Format **19**
16.1 Packet Structure . . . 19
16.2 Header Field Breakdown . . . 19
16.3 Echo Metadata Extraction . . . 19
16.4 Spoke Sub-Header . . . 19
16.5 Multi-Spoke Packing . . . 19
16.6 Azimuth and Heading . . . 19
16.7 Typical Packet Characteristics . . . 19

17 Echo Encoding Types **20**
17.1 Encoding 0: Raw Uncompressed . . . 20
17.2 Encoding 1: RLE with Repeat Marker . . . 20
17.3 Encoding 2: Delta RLE with Previous Spoke Reference . . . 20
17.4 Encoding 3: 2-Bit RLE with Dual Reference . . . 20
17.5 Encoding Selection . . . 20
17.6 Display Value Conversion . . . 20
17.7 Rotation Detection . . . 20

18 Doppler Processing **21**
18.1 Status Field Encoding . . . 21

18.2 Echo Byte Encoding . . . 21
18.3 Downstream Usage . . . 21
18.4 Practical Notes . . . 21

19 Dual-Pulse Interleaving 22
19.1 Pulse Types . . . 22
19.2 Range Dependence . . . 22
19.3 Rez Boost Effect . . . 22
19.4 Spoke Handling . . . 22

IV Network & Wire Protocol 23

20 Network Architecture 23
20.1 Topology . . . 23
20.2 Port Map . . . 23
20.3 Auto-Discovery and Network Configuration . . . 23
20.3.1 Platform-Specific IP Alias . . . 23

21 Control Protocol 24
21.1 Message Format . . . 24
21.2 Binary Keepalive . . . 24
21.3 Binary vs ASCII Disambiguation . . . 24
21.4 Response Types . . . 24
21.5 Response Parsing Pipeline . . . 24
21.6 Command Acknowledgment . . . 24
21.7 Control Channel Drain . . . 24
21.8 Initialization Sequence . . . 24
21.8.1 Phase 1: Discovery . . . 24
21.8.2 Phase 2: Authentication . . . 24
21.8.3 Phase 3: Control Channel . . . 24
21.8.4 Phase 4: Firmware Query . . . 24
21.8.5 Phase 5: Register Dump . . . 24
21.8.6 Phase 6: Steady State . . . 24

22 Command Reference 25
22.1 TX State (`$S69`) . . . 25
22.2 Range (`$S62`) . . . 25
22.3 Gain (`$S63`) . . . 25
22.4 Sea Clutter (`$S64`) . . . 25
22.5 Rain Clutter (`$S65`) . . . 25
22.6 Pulse Width (`$S68`) . . . 25
22.7 STC Near (`$S85`) . . . 25
22.8 STC Middle (`$S86`) . . . 25
22.9 STC Far (`$S87`) . . . 25
22.10 Main Bang Suppression (`$S83`) . . . 25
22.11 Antenna Height (`$S84`) . . . 25
22.12 Antenna Revolution (`$S89`) . . . 25

22.13Interference Rejection (`$S67`, param[2]=2) . . . 25
22.14Noise Reduction (`$S67`, param[2]=3) . . . 25
22.15Rez Boost (`$SEE`) . . . 25
22.16Bird Mode (`$SED`) . . . 25
22.17Doppler / Target Analyzer (`$SEF`) . . . 25
22.18No-Transmit Sectors (`$S77`) . . . 25
22.19Query Commands . . . 25

23 Periodic Telemetry **26**
23.1 Telemetry Message Table . . . 26
23.2 Telemetry Processing . . . 26
23.3 Parsed vs Discarded Data . . . 26

24 Proprietary Library Reference **27**
24.1 Module Breakdown . . . 27
24.2 Shared Command IDs . . . 27

Part I

Overview & Usage

1 Introduction

This document is the complete technical reference for the Furuno DRS4D-NXT marine radar system as interfaced through the `furuno-server` Rust application. It covers radar operation via a browser-based web interface, the internal software architecture, echo data encoding and decoding, and the full wire protocol. The intended audiences are new users learning to operate the system, contributors modifying the codebase, and developers building their own applications on top of the radar hardware.

1.1 Hardware Overview

The Furuno DRS4D-NXT is a solid-state Doppler marine radar operating in the X-band at 9.41 GHz. The scanner unit contains the antenna, transmitter, and receiver in a single sealed radome enclosure. It connects to a host processor—in this deployment, a Raspberry Pi—via a standard Ethernet cable. The scanner is assigned a fixed IP address of `172.31.3.226` on a `172.31.0.0/16` subnet.

The radar transmits at up to 25 watts using solid-state amplification rather than a traditional magnetron, which eliminates warm-up time and provides consistent performance across the operating temperature range. The antenna rotates at either 24 or 48 RPM depending on the selected speed mode. Echo data is delivered as radial spokes, with 8192 unique azimuth positions per full rotation and up to 868 range bins per spoke. The system supports six pulse width settings (S1, S2, M1, M2, M3, L), 18 range scales from 1/16 NM to 48 NM, and built-in Doppler target discrimination.

1.2 Software Components

The system consists of three software components.

furuno-server is a Rust binary that implements the complete Furuno wire protocol from scratch. It handles network discovery, authentication, control commands, echo data reception and decoding across four encoding types, PPI (Plan Position Indicator) rendering, and serves a web interface for browser-based operation. The application is built with Axum 0.7 for the HTTP server and Tokio for the async runtime. All radar communication—TCP control, UDP multicast echo reception, and subnet management—is handled natively without any Furuno proprietary libraries.

The *web interface* is a set of static HTML, CSS, and JavaScript files served by the Rust application. It provides four pages: a controls dashboard for quick radar operation, a full-screen PPI display with history compositing, a detailed settings panel, and a system status view. The interface communicates with the server through a REST API and polls for updated radar images at 500 ms intervals.

The *documentation* consists of this technical manual and inline source code comments. The manual is maintained as a single self-contained LaTeX file with all diagrams rendered via TikZ.

1.3 Reverse Engineering Context

All protocol knowledge in this document was obtained through reverse engineering. The primary source was the proprietary Furuno shared library `libNAVNETDLL.so`, which was analyzed through symbol table inspection, function call tracing, and behavioral observation. Live packet captures between the library and the radar hardware provided ground truth for the authentication handshake, control message format, and echo data encoding. The Rust implementation in `furuno-server` was built incrementally by reproducing each protocol phase and validating against the radar's actual responses.

The original C++ application that used `libNAVNETDLL.so` directly served as a reference implementation during this process. Command IDs, parameter ranges, and encoding schemes documented here have been verified against both the disassembled library and live radar behavior. Where the proprietary library's behavior was ambiguous or undocumented, the values in this manual reflect what was observed to work reliably in practice.

2 Quick Start & Installation

2.1 Prerequisites

The Rust toolchain is required to build the application. Install it via `rustup` if it is not already present.

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source ~/.cargo/env
```

The application builds and runs on Linux (x86_64 and aarch64), macOS, and Windows. The primary deployment target is a Raspberry Pi running Debian, connected to the radar via its Ethernet port. No external C libraries or Furuno SDKs are required.

2.2 Building

From the repository root, build the release binary.

```
cd furuno-server
cargo build --release
```

The compiled binary is located at `furuno-server/target/release/furuno-server`. A release build is recommended for deployment as it enables compiler optimizations that significantly improve echo decoding and PPI rendering throughput.

2.3 First Run

Start the web server with the `web` mode.

```
cargo run --release -- web
```

The web interface is then available at `http://localhost:8080` in any browser. On startup, the application attempts to discover the radar automatically (see below). If the radar is not reachable, the web server starts anyway and serves a degraded interface. The `/reconnect` endpoint can be used to retry the connection later.

2.4 Auto-Discovery

The application resolves the radar and host IP addresses through a three-stage fallback process.

First, it checks for explicit command-line flags. If `--radar-ip` is provided, that address is used directly. The `--host-ip` flag is optional; if omitted, the application determines a suitable host address on the radar's subnet automatically.

```
cargo run --release -- --radar-ip 172.31.3.226 web
```

Second, if no flags are provided, the application broadcasts UDP discovery beacons on port 10010 across all detected network interfaces and listens for 3 seconds. If the radar responds, both the radar and host addresses are extracted from the response.

Third, if discovery fails, the application falls back to the default addresses: `172.31.3.226` for the radar and `172.31.1.100` for the host. If the host does not already have an interface on the `172.31.0.0/16` subnet, a secondary IP alias is added to the physical Ethernet interface automatically. On Linux this uses `ip addr add`; on macOS, `ifconfig alias`.

3 CLI Modes

3.1 General Syntax

The application accepts optional IP flags followed by a mode name and mode-specific arguments.

```
furuno-server [--radar-ip IP] [--host-ip IP] <mode> [args...]
```

The two user-facing modes are `web` and `discover`. Additional developer-oriented modes (`handshake`, `tx-on`, `tx-off`, `echo`, `full`, `render-live`, `dump-raw`, `test-commands`, `validate`) are documented in §13. If no mode is given, the application defaults to `handshake`.

3.2 Web Server Mode

The `web` mode starts the HTTP server and is the primary way to operate the radar.

```
furuno-server web [port] [webdir]
```

The optional `port` argument sets the listening port (default 8080). The optional `webdir` argument specifies the path to the static web files (default `../web` relative to the binary). On startup, the server attempts to authenticate with the radar and initialize the control channel. If the radar is unreachable, the server starts anyway with degraded functionality—the PPI display shows a blank image and setting changes are queued until a connection is established via the `/reconnect` endpoint.

3.3 Discover Mode

The `discover` mode broadcasts UDP beacons on port 10010 across all network interfaces and prints any radar responses it receives.

```
furuno-server discover
```

This is useful for determining the radar's IP address on an unfamiliar network or verifying that the radar is powered on and reachable. The mode runs until a response is received or the timeout expires.

4 Web Interface

4.1 Starting the Server

4.2 Page Tour

4.2.1 Controls (index.html)

4.2.2 Radar View (radar.html)

4.2.3 Settings (settings.html)

4.2.4 System Status (status.html)

4.3 PPI Display

5 Recording & Data Formats

5.1 Starting and Stopping Recording

5.2 Output Directory Structure

5.3 CSV Format

5.4 Settings Snapshot

6 Pattern System

6.1 What Patterns Do

6.2 Simple Pattern Format (7-Field)

6.3 Advanced Pattern Format (13-Field)

6.4 Pattern Queue

6.5 Loading Patterns

6.6 NaN Override Behavior

Part II

Software Internals

7 System Architecture Overview

7.1 Module Summary

7.2 System-Level Block Diagram

7.3 Module-Level Diagram

7.4 Data Flow Diagram

7.5 Concurrency Model

8 PPI Rendering

8.1 Coordinate Transformation

8.2 Color Mapping

8.3 Overlay Elements

8.4 PNG Encoding

8.5 How to Modify

9 Web Server

9.1 Framework

9.2 Shared State

9.3 Route Registration

9.4 Handler Pattern

9.5 Echo Listener Thread

9.6 Static File Serving

9.7 How to Add a New Endpoint

10 REST API Reference

10.1 Power and Connection

10.2 Settings

10.3 State Queries

10.4 Echo Image

10.5 Recording

10.6 Pattern Execution

10.7 Stub Endpoints

11 Recording System Internals

11.1 Echo Listener Integration

11.2 Directory Creation

11.3 CSV Writing

11.4 Settings Snapshots

11.5 Setting Change History

12 Pattern Execution Engine

12.1 Step Parsing

12.2 Differential Command Generation

12.3 Single Pattern Execution

12.4 Queue Execution

12.5 NaN Override Logic

12.6 Stop Mechanism

13 CLI Modes (All Other Modes)

13.1 Handshake

13.2 TX On / TX Off

13.3 Full Cycle

13.4 Render Live

13.5 Echo Listen

13.6 Command Test

13.7 Dump Raw

13.8 Test Commands

13.9 Validate

14 Connection State Machine

14.1 States and Transitions

14.2 RadarError Enum

14.3 Timeout Values

14.4 Graceful Degradation

14.5 Keepalive Handling

15 Testing & Validation

15.1 Running Tests

15.2 Test Categories

15.2.1 Connection Tests

15.2.2 TX Tests

15.2.3 Settings Tests

15.2.4 Echo Impact Tests

15.2.5 Render Integration Test

15.3 A/B Command Testing

15.4 Validation Suite

Part III

Echo Data Pipeline

16 Echo Data Format

16.1 Packet Structure

16.2 Header Field Breakdown

16.3 Echo Metadata Extraction

16.4 Spoke Sub-Header

16.5 Multi-Spoke Packing

16.6 Azimuth and Heading

16.7 Typical Packet Characteristics

17 Echo Encoding Types

17.1 Encoding 0: Raw Uncompressed

17.2 Encoding 1: RLE with Repeat Marker

17.3 Encoding 2: Delta RLE with Previous Spoke Reference

17.4 Encoding 3: 2-Bit RLE with Dual Reference

17.5 Encoding Selection

17.6 Display Value Conversion

17.7 Rotation Detection

18 Doppler Processing

18.1 Status Field Encoding

18.2 Echo Byte Encoding

18.3 Downstream Usage

18.4 Practical Notes

19 Dual-Pulse Interleaving

19.1 Pulse Types

19.2 Range Dependence

19.3 Rez Boost Effect

19.4 Spoke Handling

Part IV
Network & Wire Protocol

20 Network Architecture

20.1 Topology

20.2 Port Map

20.3 Auto-Discovery and Network Configuration

20.3.1 Platform-Specific IP Alias

21 Control Protocol

21.1 Message Format

21.2 Binary Keepalive

21.3 Binary vs ASCII Disambiguation

21.4 Response Types

21.5 Response Parsing Pipeline

21.6 Command Acknowledgment

21.7 Control Channel Drain

21.8 Initialization Sequence

21.8.1 Phase 1: Discovery

21.8.2 Phase 2: Authentication

21.8.3 Phase 3: Control Channel

21.8.4 Phase 4: Firmware Query

21.8.5 Phase 5: Register Dump

21.8.6 Phase 6: Steady State

22 Command Reference

22.1 TX State ($S69)

22.2 Range ($S62)

22.3 Gain ($S63)

22.4 Sea Clutter ($S64)

22.5 Rain Clutter ($S65)

22.6 Pulse Width ($S68)

22.7 STC Near ($S85)

22.8 STC Middle ($S86)

22.9 STC Far ($S87)

22.10 Main Bang Suppression ($S83)

22.11 Antenna Height ($S84)

22.12 Antenna Revolution ($S89)

22.13 Interference Rejection ($S67, param[2]=2)

22.14 Noise Reduction ($S67, param[2]=3)

22.15 Rez Boost ($SEE)

22.16 Bird Mode ($SED)

22.17 Doppler / Target Analyzer ($SEF)

22.18 No-Transmit Sectors ($S77)

22.19 Query Commands

23 Periodic Telemetry

23.1 Telemetry Message Table

23.2 Telemetry Processing

23.3 Parsed vs Discarded Data

24 Proprietary Library Reference

24.1 Module Breakdown

24.2 Shared Command IDs